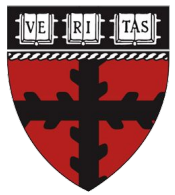
# Lecture 10: Connecting to the Cloud: Networking, Authentication & Storage

AC215/E115

Pavlos Protopapas
SEAS/Harvard



# Outline

- Networking & Ports
- Security and authentication (detour)
- Google Cloud Storage
- Connecting to Google Drive
- Tutorial

# Outline

- **Networking & Ports**
- Security and authentication (detour)
- Google Cloud Storage
- Connecting to Google Drive
- Tutorial

# Networking & Ports — What Was All That?

**In Containers II you saw these. What was actually going on?**

**Two numbers** — why?

```
$ docker run -p 8090:9100 gateway-img
  → http://localhost:8090

  GET http://processor:9200
```

**Which machine** is that?

**A name** — where's the address?

**We'll build it up — network, then port — answer each one, and end on the real question: who's allowed in?**

# What Is a Network? (IP = a Street Address)

## A house = anything with an IP — even a container. Four flavors of IP:

### Private IP — a Docker network

`app-net · 172.18.0.x`



**gateway**
`172.18.0.2`



**processor**
`172.18.0.3`

### Public IP — the open internet



**google.com**
`142.250.4.100`



**your-app.com**
`34.120.10.5`

### Private IP — your home Wi-Fi

`home Wi-Fi · 192.168.1.x`



**MacBook**
`192.168.1.10`



**iPad**
`192.168.1.11`



**iPhone**
`192.168.1.12`

### Loopback — every machine has one



**localhost**
`127.0.0.1`

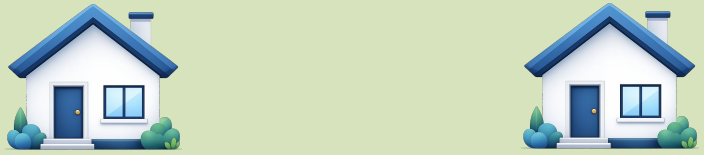
`127.0.0.1` is nicknamed `localhost`. Every house above has its own —always meaning **"this machine, itself."**

# What Is a Network? (IP = a Street Address)

**A house = anything with an IP — even a container. Four flavors of IP:**

## Private IP — a Docker network

`app-net · 172.18.0.x`



## Public IP — the open internet



**One machine — or a container — can hold several IPs at once.**
Your laptop is both `127.0.0.1` (itself) and its Wi-Fi IP; a container gets one per Docker network it joins; a cloud server has a **private** IP and optionally a **public** one.

## Priva… one

`home`



**MacBook**
`192.168.1.10`



**iPad**
`192.168.1.11`



**iPhone**
`192.168.1.12`



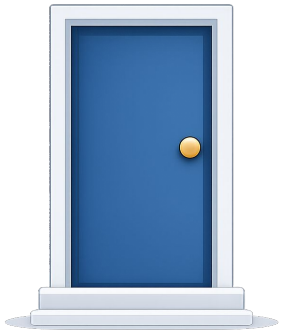
**localhost**
`127.0.0.1`

`127.0.0.1` is nicknamed **`localhost`**. Every house above has its own —always meaning **"this machine, itself."**

# What Is a Port? (a Door of the House)

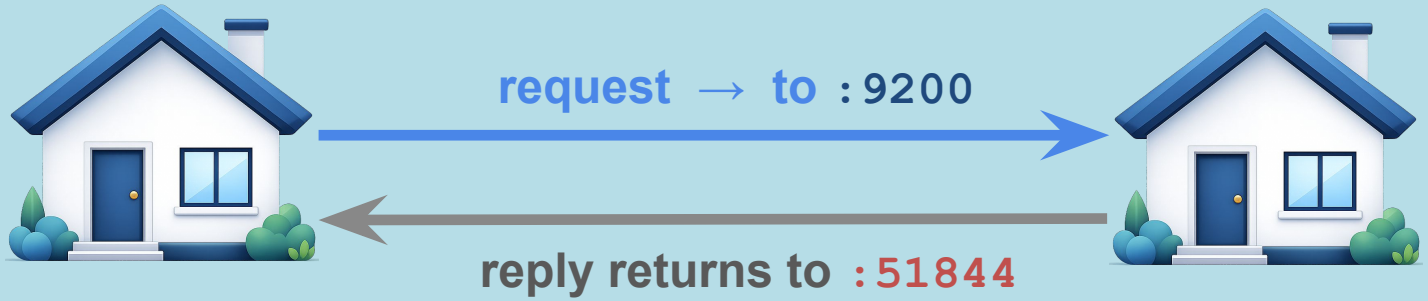
**A machine runs many services — the port picks which one. Full address = IP : port.**

**the idea:** a door = a port



`:9200`
a service listens here
= its inbound port

**one connection** · out to the service, then the reply comes back




**gateway**
`172.18.0.2 : 51844`
client · random, outbound

**processor**
`172.18.0.3 : 9200`
server · listening, inbound

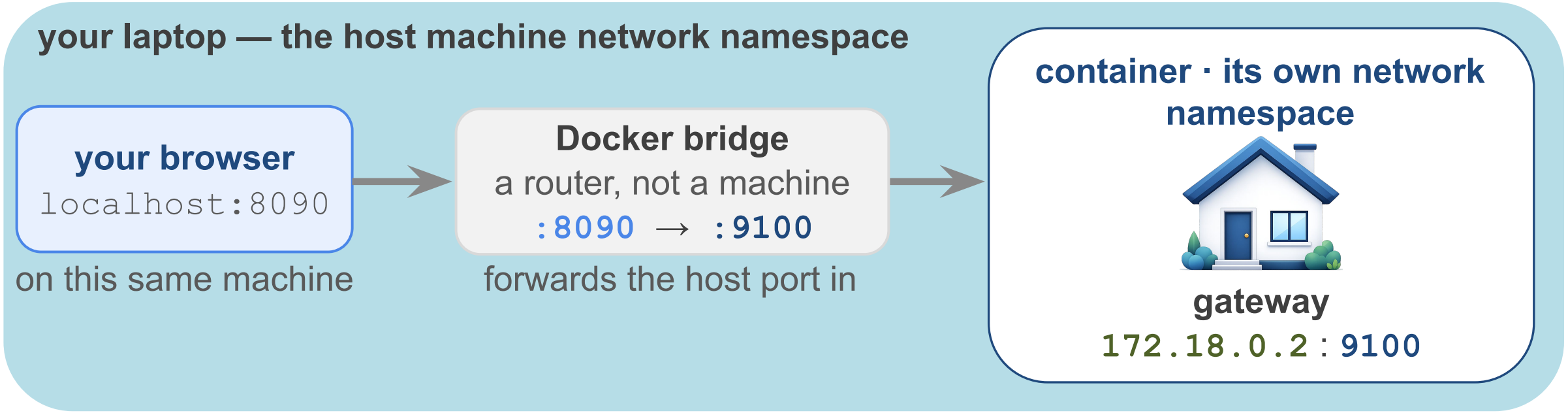
**Inbound (listening) port** — fixed: the service waits at `:9200`, the number you publish.
**Outbound port** —**random** (`:51844`); the reply returns to that same port.

# What Is a Port?  (a Door of the House)

| Port range | What it is | Examples |
|---|---|---|
| **0 – 1023** | Well-known · system (reserved) | 22 SSH · 80 HTTP · 443 HTTPS |
| **1024 – 49151** | Registered · pick yours here | 9200 · 8080 · 3000 · 5432 |
| **49152 – 65535** | Dynamic · ephemeral (outbound) | :51844  (the OS picks it) |

# Port Publishing, Revisited (the Tunnel)

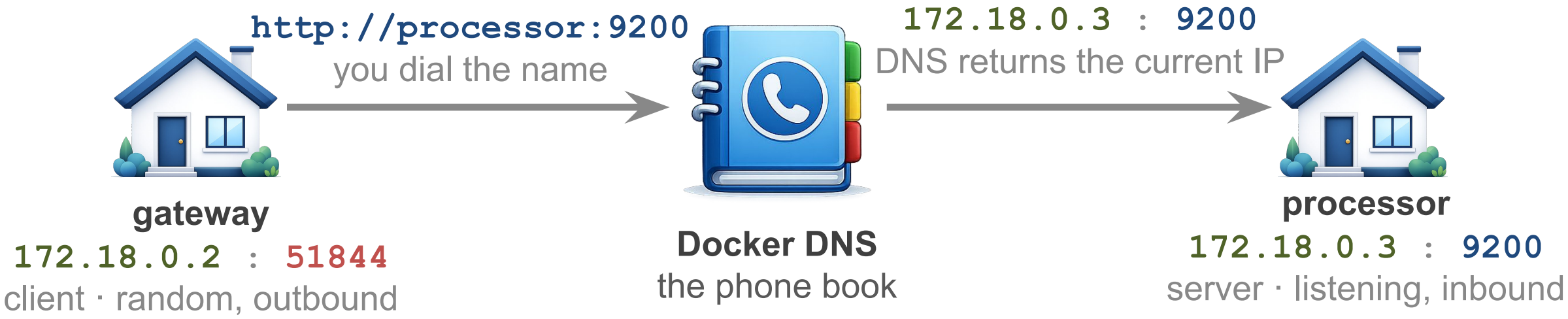
**`-p 8090:9100` tunnels your laptop's door → the app's door in the container.**




- **Its own namespace** — a container's ports live in its own network namespace (Containers I). Its `:9100` is sealed off from your laptop until you publish it.
- **The two numbers** — **HOST** (8090, **you pick**) : **CONTAINER** (9100, **where the app listens**).

# Service-to-Service by Name, Revisited

**Why `http://processor:9200` worked — Docker gives every service a name.**




- **Name = hostname** — on a shared network, each service name is addressable.
- **Docker's DNS resolves it** to the container's current IP. IPs change on restart; the **name is stable** — so always use the name.
- **Compose wires it automatically** — the service name is the hostname. That's "Networking for Free."

# Why This Matters for the Cloud

**Same ideas — just bigger, and now on a public street.**





| | On your laptop | In the cloud |
|---|---|---|
| **Address** | localhost / private IP | a public IP + a DNS name |
| **Who can reach it** | not on the internet — your router keeps the world out | on the internet — a firewall must open the port (closed by default) |
| **Getting traffic in** | you publish the container's port: -p 8090:9100 | a load balancer out front routes traffic in |

A public address + an open port = anyone on the internet can knock.

**The question flips — not "how do I reach it?" but "who's allowed in?"**

→ **Authentication (next)**

# Outline

- Networking & Ports
- **Security and authentication (detour)**
- Google Cloud Storage
- Connecting to Google Drive
- Tutorial

# Why Authentication Matters in MLOps

- A public, unprotected API endpoint = risk:
  - Runaway cloud bills from LLM abuse
  - Data leaks or malicious use of your AI
- First defense: **authentication & access management**
- Goal: ensure only **authorized users & services** interact with your AI

# How Authentication Works: 3 Categories (Factors)



FACTOR 1

## Something You Know

- Passwords, PINs
- Security questions **(weak)**



FACTOR 2

## Something You Have

- Authenticator apps (tokens)
- Hardware keys (YubiKey)
- Phone (SMS/call codes)
- Digital certificates



FACTOR 3

## Something You Are

- Fingerprint
- Facial recognition
- Iris/retina scan
- Voice recognition

# MFA, Biometrics, and Passwordless

**MFA combines two or more different factors**
Password + authenticator app = know + have.

**Biometrics can be part of MFA**
Passkey on your device + fingerprint = have + are.

**Passwordless can also be MFA**
The same passkey + fingerprint uses two factors, with no password.

# Authentication in Google Cloud

**Console sign-in: your Google account**
Use a password with a second step, or a passkey.

**Second step: something you have**
An authenticator code, Google prompt, or security key.

**VM access: SSH keys or certificates**
Your SSH client proves possession of a private key.

# API Keys: Basic Way to Call an API

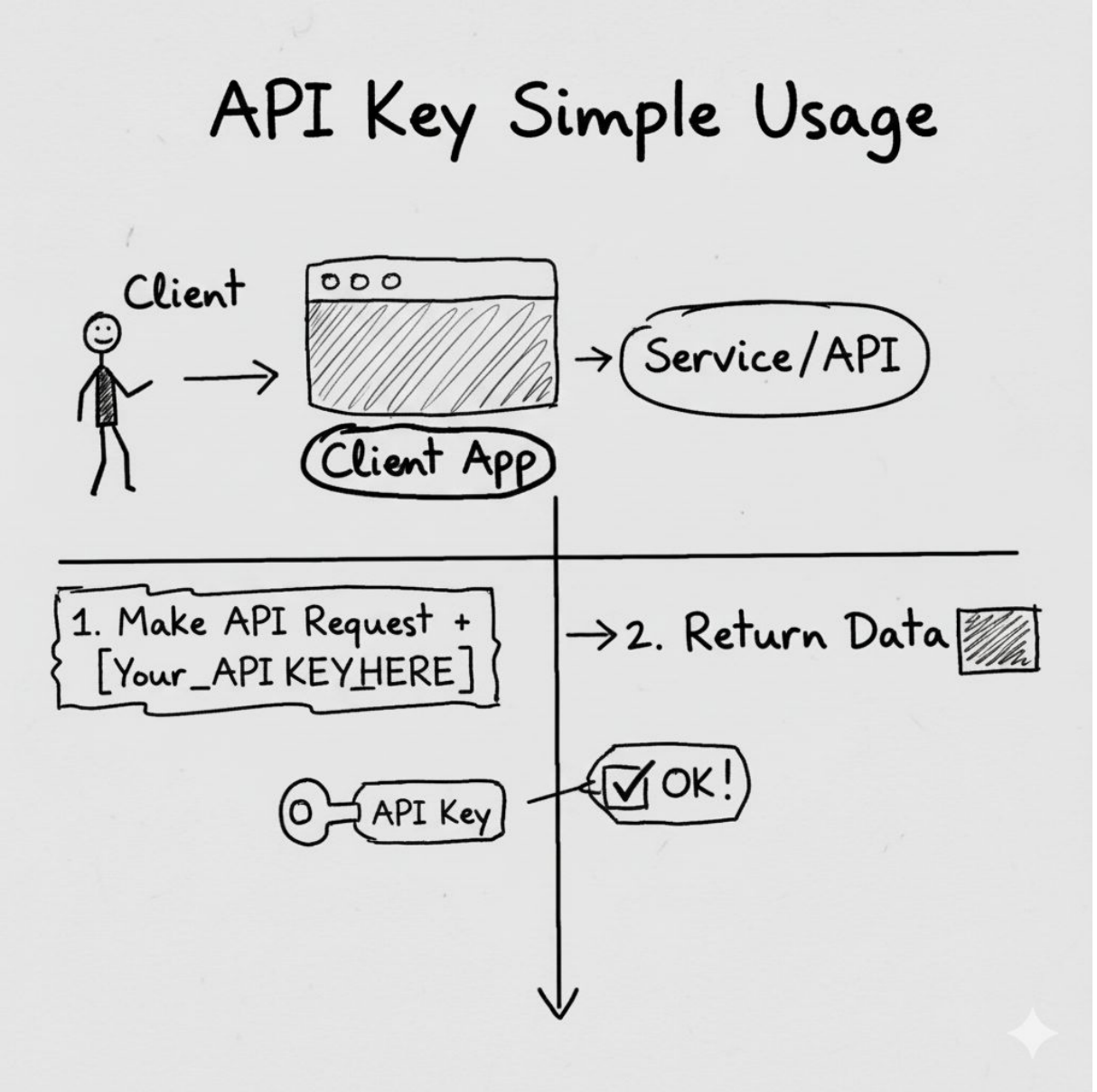
**API Key** 🔑

- APIs are services that programs call to get something done (like translation or weather data).
- To use them, we need a **token** — a small secret string of letters and numbers.
- The token tells the API *who* is calling. It's used both to **authenticate** (prove you're allowed in) and to **control resources** (for example, limit how many calls you can make per day).

**Example:**

We put a translation model online as an API. Each caller gets a token. The token doesn't tell us much about the user, but it lets us track requests and enforce limits (say, 1,000 calls per day).

# API Keys: Basic Way to Call an API



# OAuth 2.0: User Consent for Access

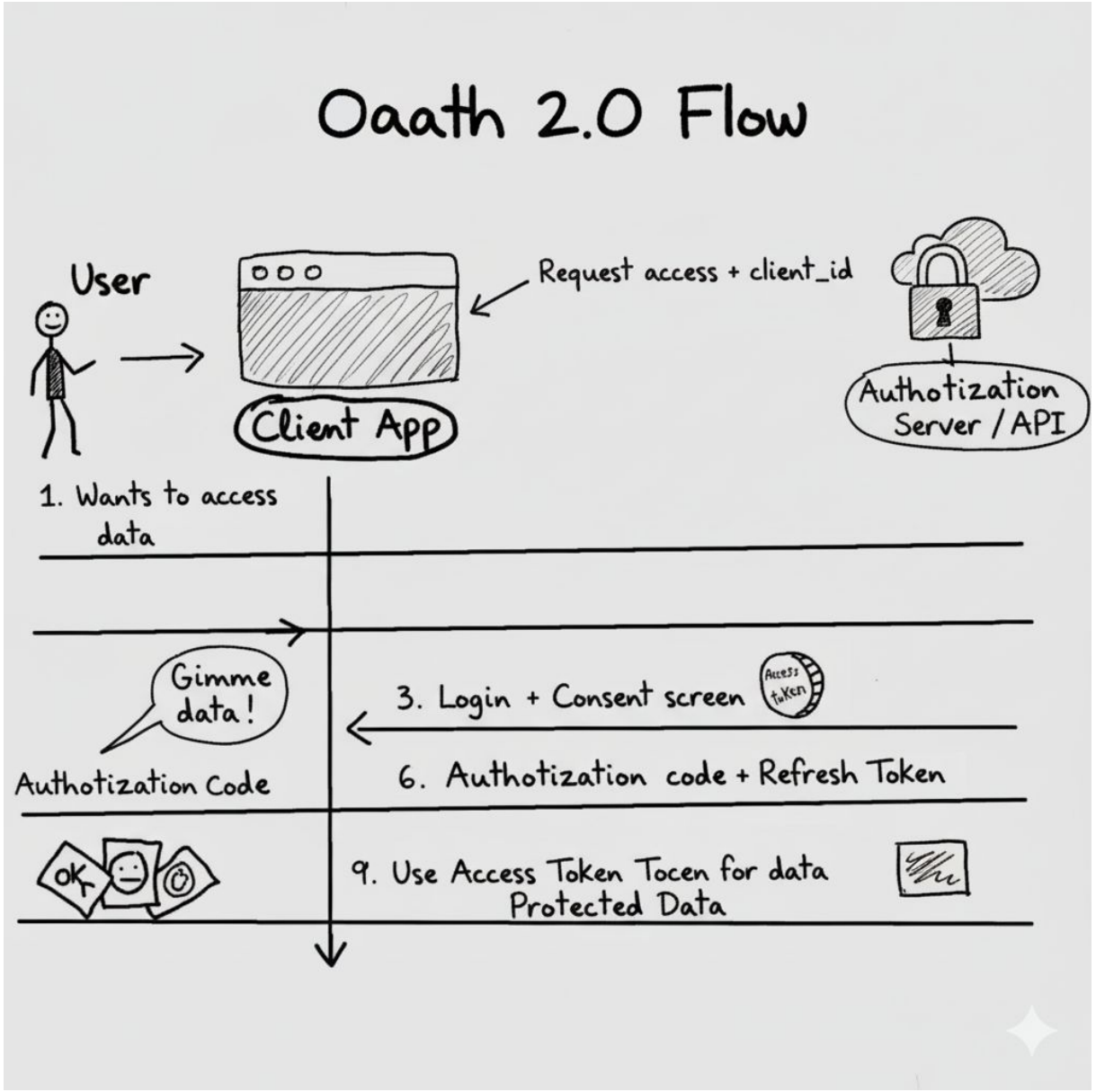
**OAuth 2.0** 🙋

- Used when an **end-user** gives permission for an app to act on their behalf.
- Instead of sharing their password, the user approves access with a simple screen (like “Sign in with Google”).
- The app then gets a **temporary token** it can use to work with the user’s data.

**Example:**

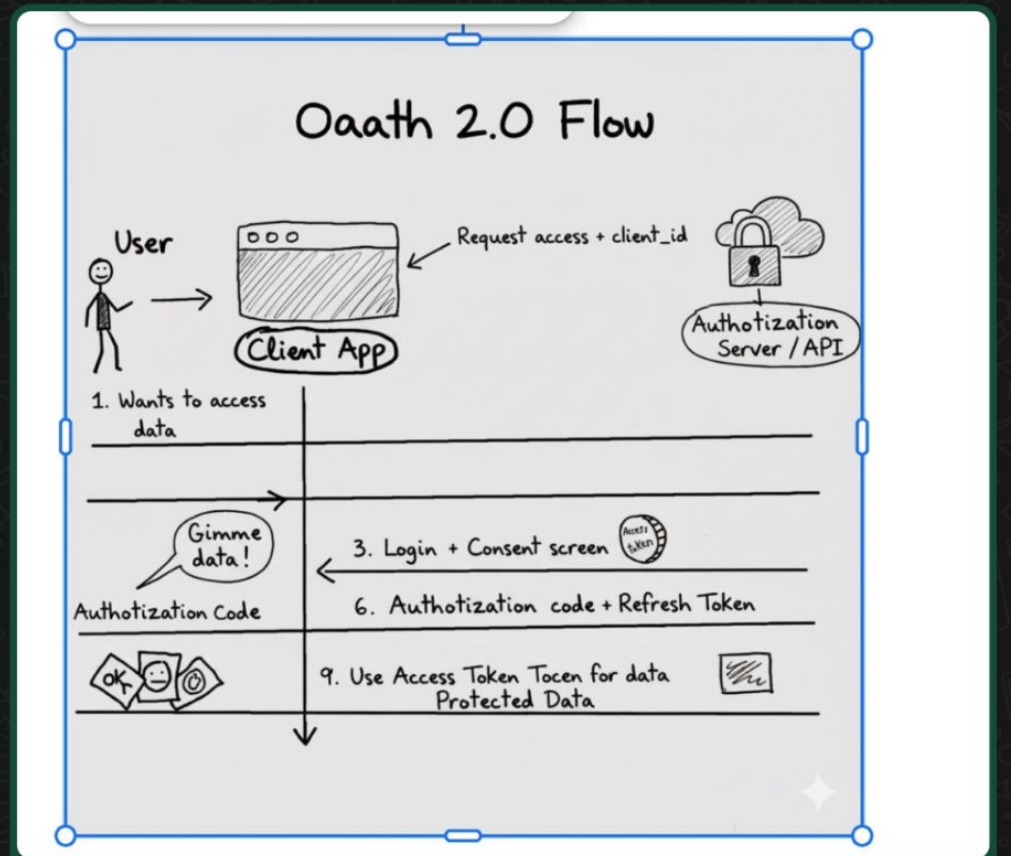
We build an app to analyze documents in Google Drive. The user signs in with Google and agrees to share their files. Our app gets a token to read those files, but we never see or store their password.

# OAuth 2.0: User Consent for Access”



# API Keys: Basic Way to Call an API

Today




This baby has step1, 3, 6 and 9 12:26 PM

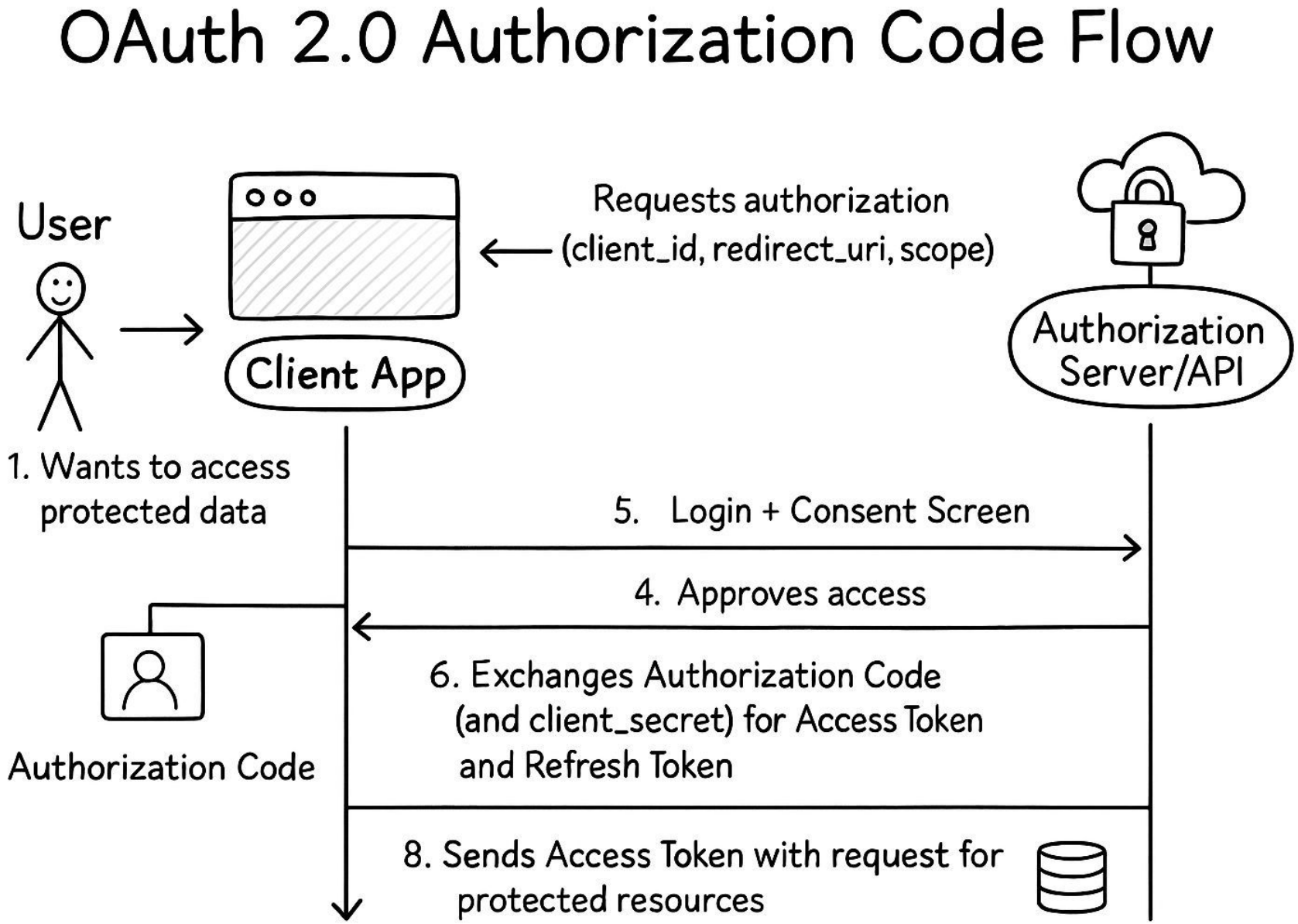
You

This baby has step1, 3, 6 and 9



Ohh crap, Gemini forgot how to count 12:27 PM

# API Keys: Basic Way to Call an API



# OAuth 2.0 Authorization Code Flow

1. App requests access to the user’s data.
2. User signs in with Google and approves the requested scopes.
3. Google redirects to the app with an authorization code.
4. App exchanges the code for an access token.
5. App sends the access token to the API to read the allowed data.

# Service Accounts: Identities for Code

**Service Accounts** 🤖

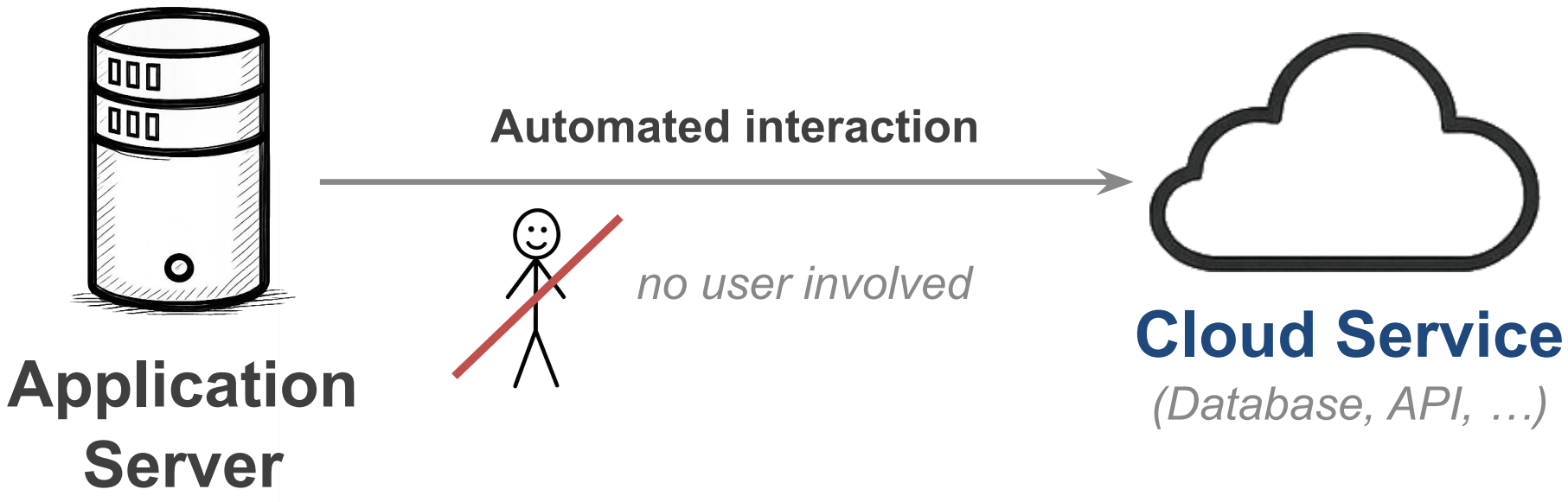
- Used for **server-to-server communication** (not people).
- Think of it as an **identity for your code**.
- Your program uses this identity to prove who it is and to get permission to access resources.

**Example:**

Suppose a script needs to copy files into a Google Cloud Storage bucket. We assign it a Service Account with storage permissions. The script uses that account to authenticate and securely upload files to the bucket — without anyone logging in.

# Service Accounts: Identities for Code

**Server-to-server: your code authenticates as its own identity — no human, no login screen.**



# Authentication vs. Authorization (IAM roles)

- **Authentication** = proving identity (user, service, or workload).
- **Authorization (IAM)** = defining *what actions* that identity can take.
- Controlled through **roles & policies**:
  - Viewer (read-only)
  - Editor (read & write)
  - Admin (full control)
- 🔑 Best practice: **least privilege** → give each account only the permissions it needs.

# Default Service Accounts: Quick Start, Not for Production

**Default Service Account ⚙️**

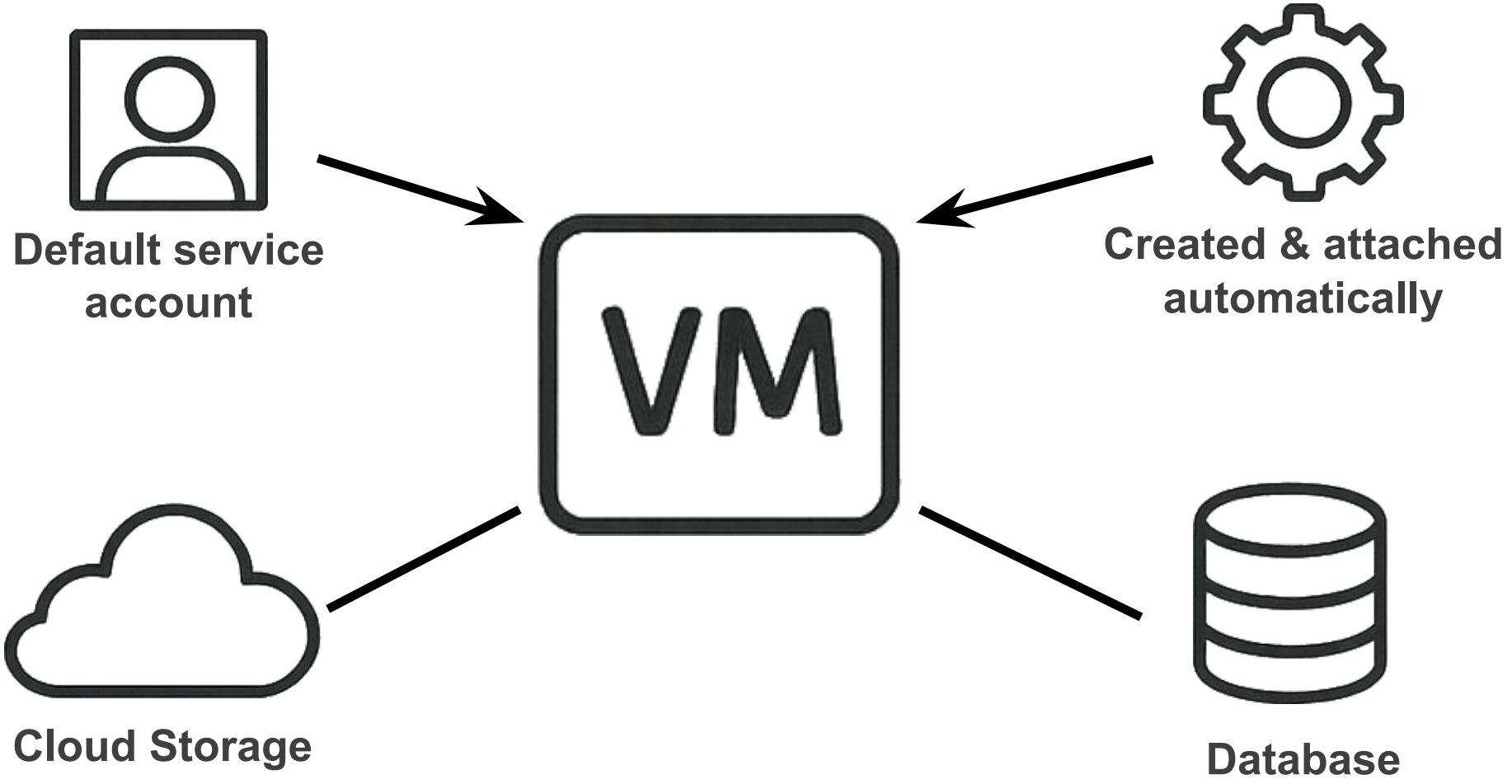
- GCP automatically gives some resources (like VMs) a **generic account** so you can start quickly.
- It's like a "starter identity" for your code.

**Example:**

When you create a new VM, it already has a Default Service Account. This means you can immediately run commands or access a Cloud Storage bucket in the same project without setting up anything extra.

⚠️ Warning: This is handy for testing, but in production you should replace it with a Service Account that has only the permissions you really need.

# Default Service Accounts: Quick Start, Not for Production



# Workload Identity Federation: Secure Access from Outside GCP

**Workload Identity Federation** 🤝

- Lets services **outside of GCP** (like AWS, Azure, GitHub Actions, or on-prem servers) securely access GCP resources.
- Removes the need to copy, store, or manage GCP service account keys.

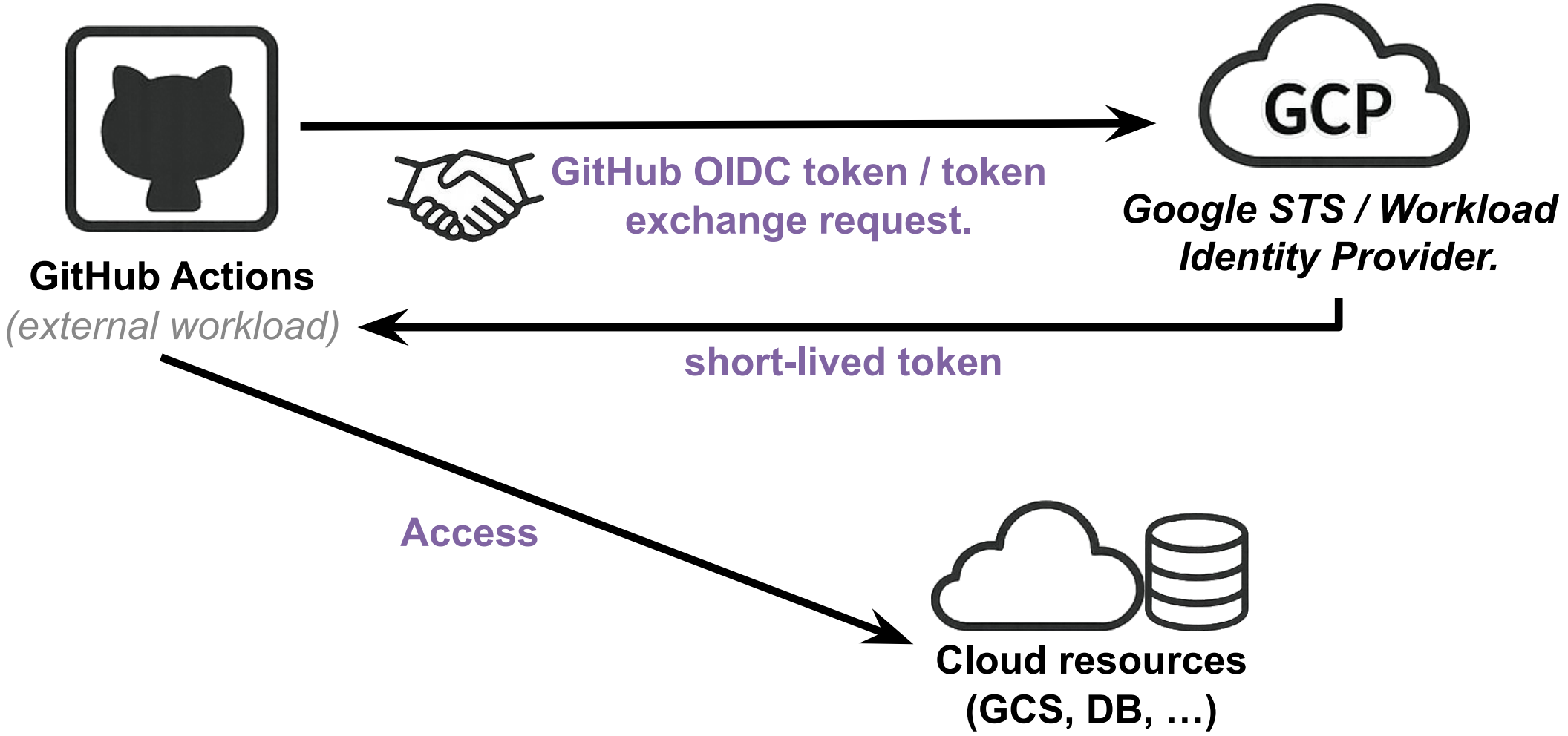
**Example:**

Our CI/CD pipeline runs on GitHub Actions (outside GCP). We set up Workload Identity Federation so GitHub can request a short-lived token from GCP. GitHub presents its own identity, GCP verifies it, and issues a temporary access token.

➡️ This way, GitHub can deploy to Cloud Run securely, without us ever copying keys around.

# Workload Identity Federation: Secure Access from Outside GCP

**External workloads (like GitHub Actions) get GCP access with short-lived tokens — no key to download or leak.**

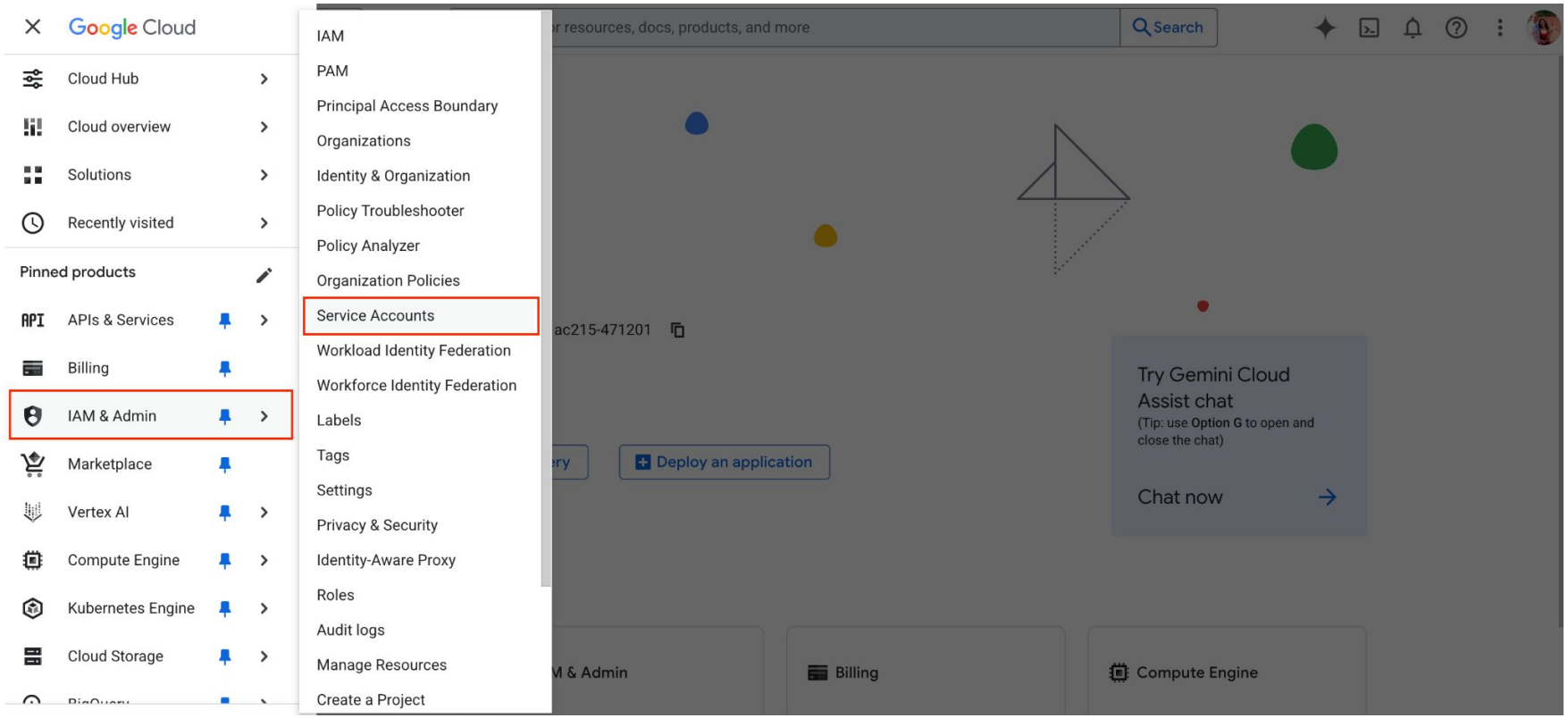


# Choosing the Right GCP Auth Tool

- **For simple, public API calls (tracking/limits):** → API Key
- **For an app acting on a user's behalf (with consent):** → OAuth 2.0
- **For a program or service running inside GCP:** → Service Account
- **For a program or service running outside GCP (AWS, GitHub Actions, on-prem servers):** → Workload Identity Federation

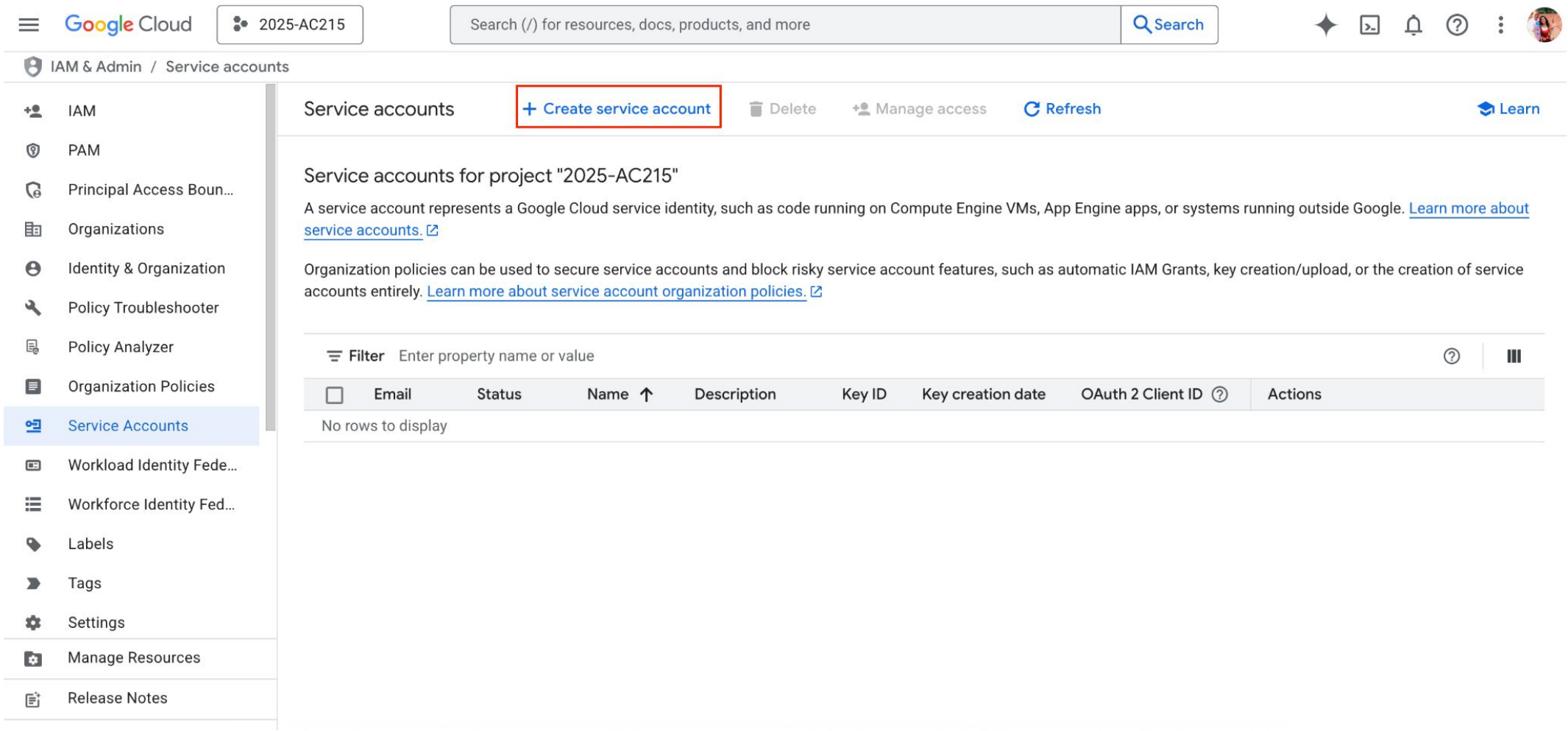
# Setup GCP Service Account

**Step1:** Go to the GCP Console, search for "Service accounts" in the top search box, or navigate to "IAM & Admin" > "Service accounts" from the top-left menu.



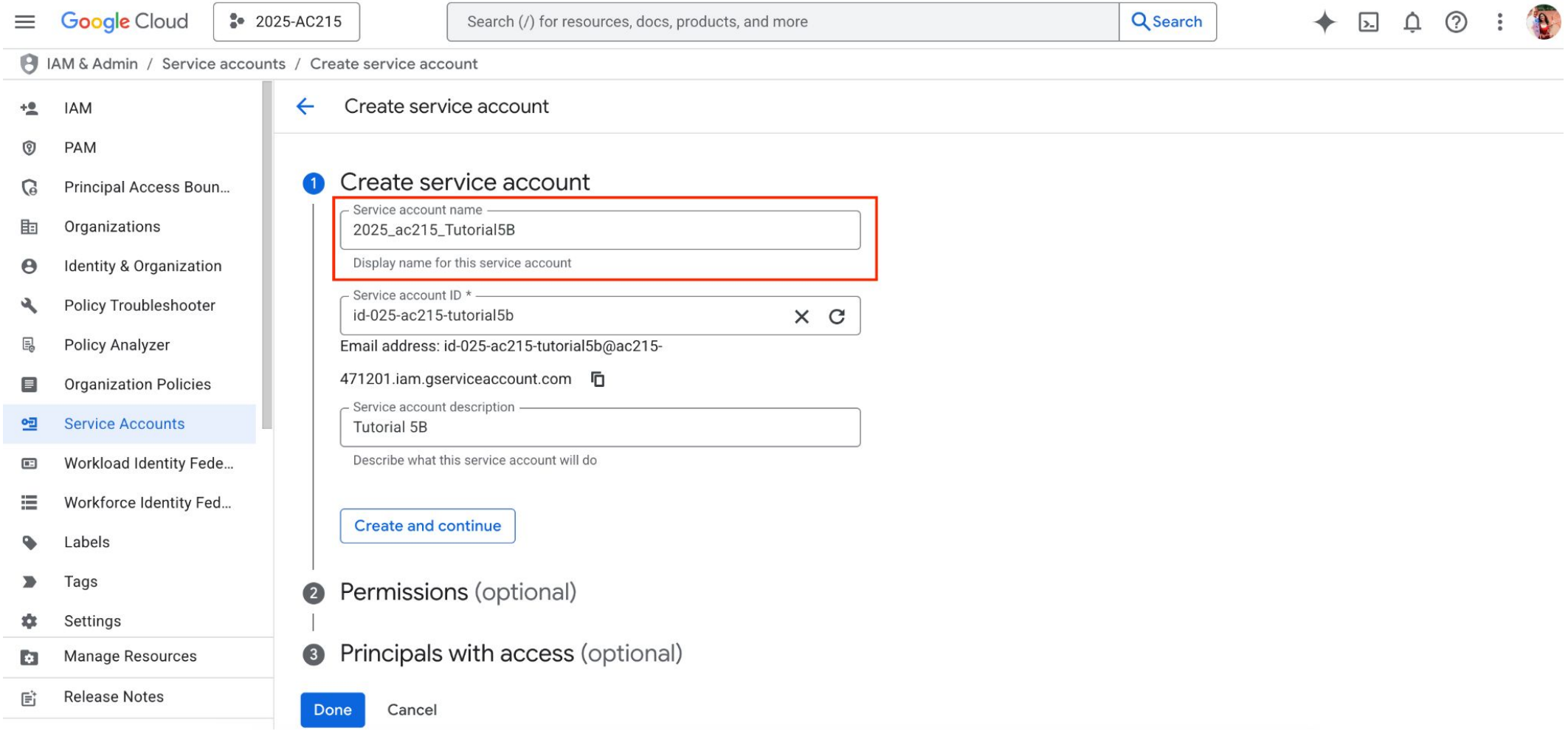
# Setup GCP Service Account

**Step2:** Create a new service account called "data-service-account."



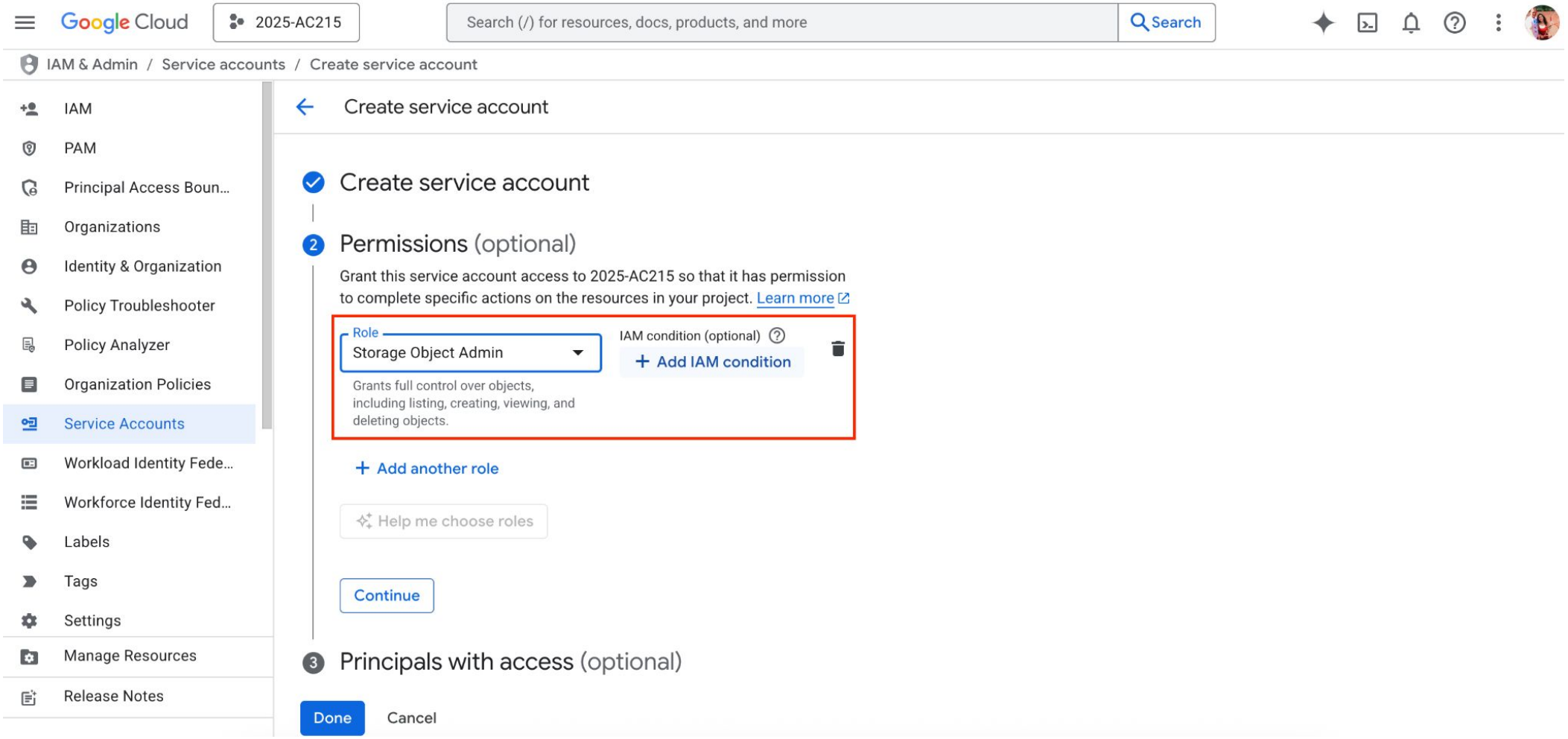
# Setup GCP Service Account

**Step2:** Create a new service account called "data-service-account."



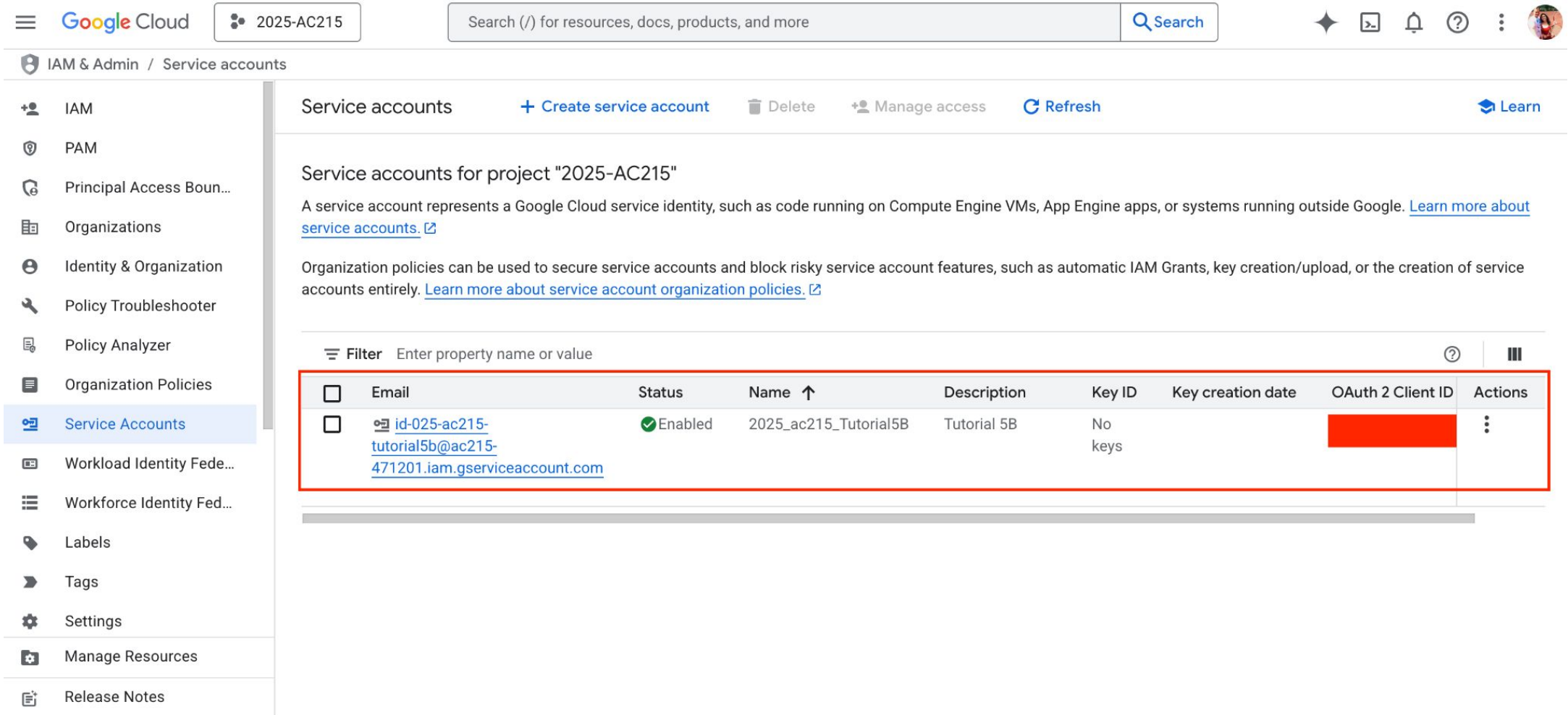
# Setup GCP Service Account

**Step3:** "Grant this service account access to project" select "Cloud Storage" > "Storage Admin".



# Setup GCP Service Account

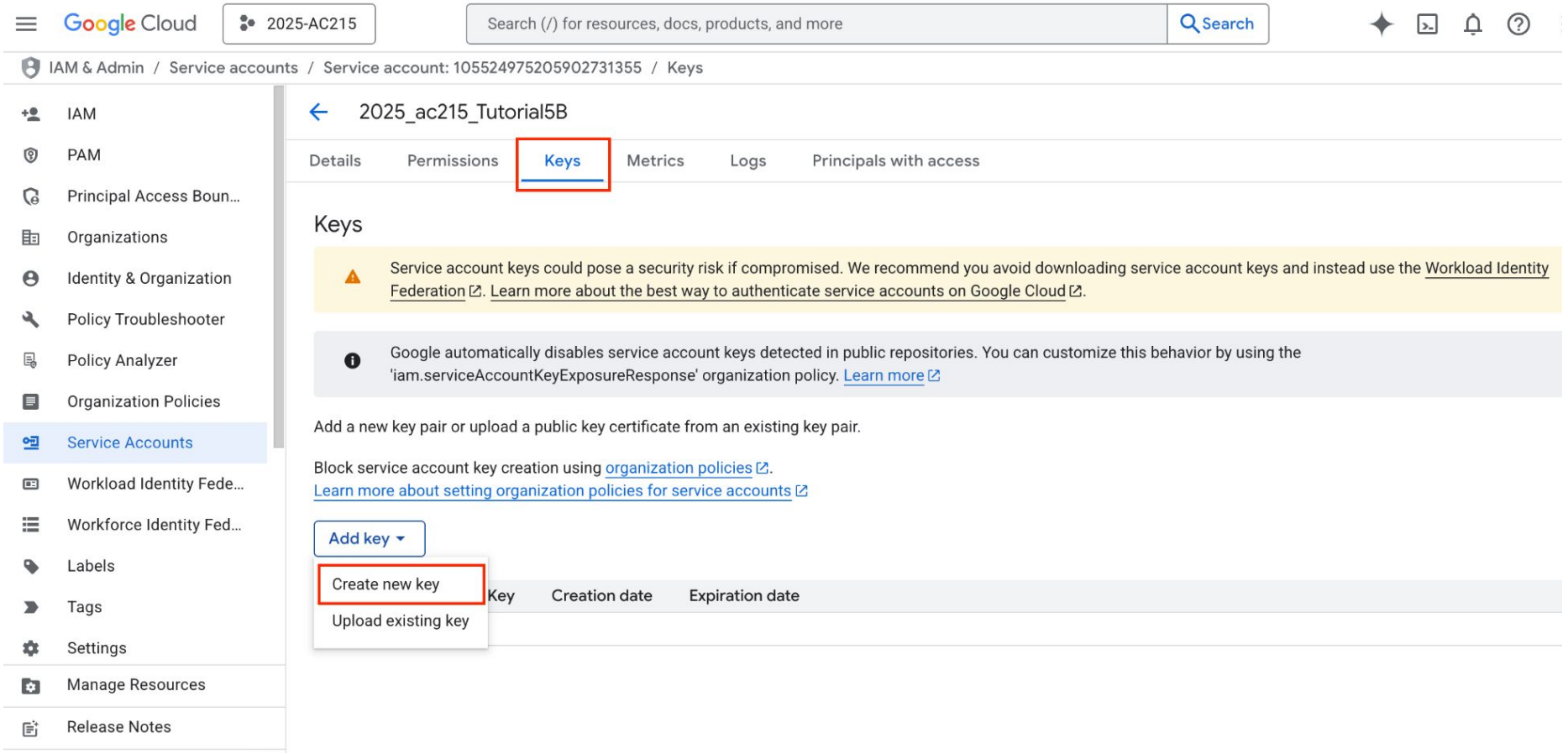
**Step3:** "Grant this service account access to project" select "Cloud Storage" > "Storage Admin".



# Setup GCP Service Account

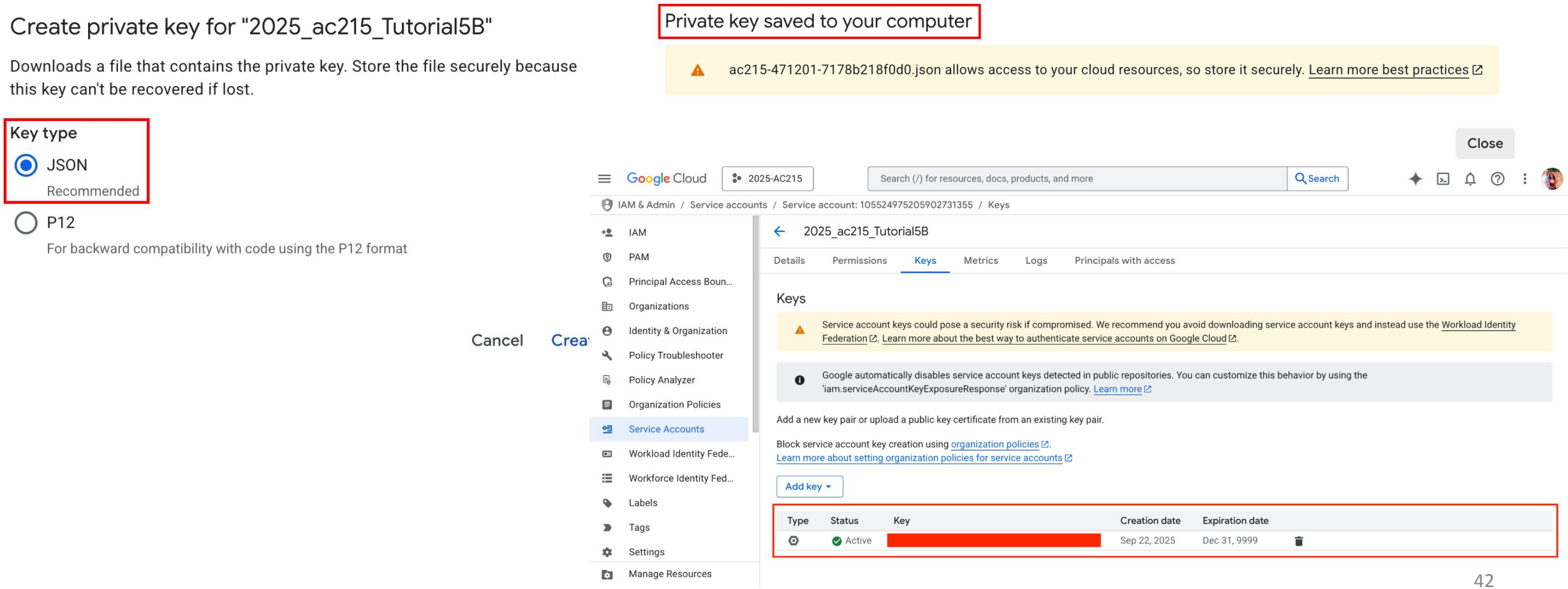
**Step4:** Click in service account and navigate to the tab "KEYS"

- Click in the button "ADD Key (Create New Key)" and Select "JSON". This will download a private key JSON file to your computer.



# Setup GCP Service Account

**Step5:** Copy this JSON file into the secrets folder and rename it to data-service-account.json.



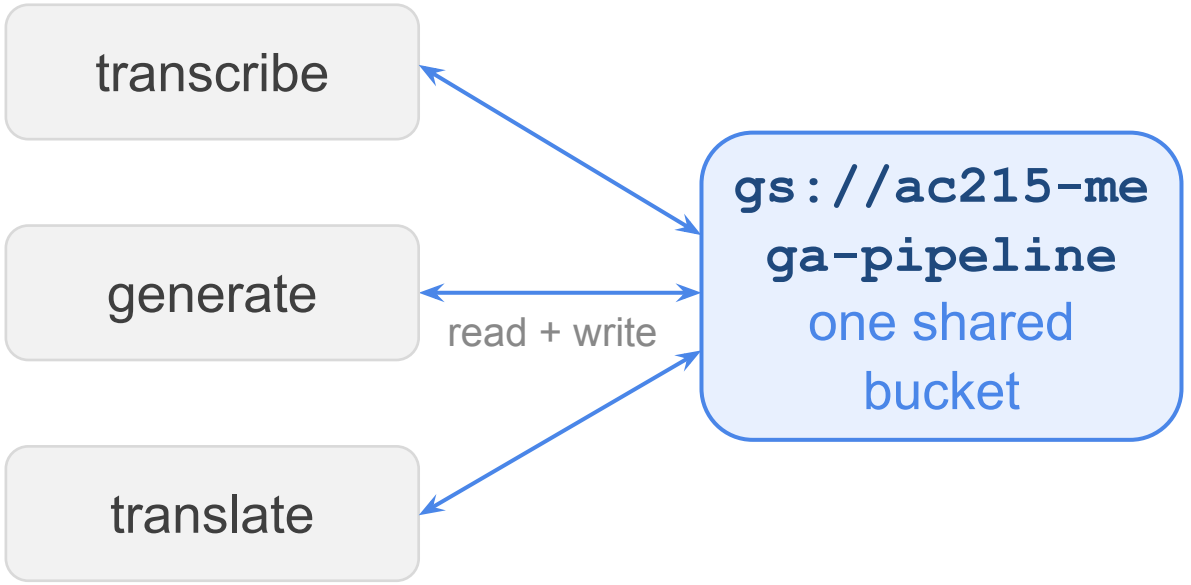
# Outline

- Networking & Ports
- Security and authentication (detour)
- **Google Cloud Storage**
- Connecting to Google Drive
- Tutorial

# What is Google Cloud Storage?

**Cloud object storage — one durable store every container can reach.**

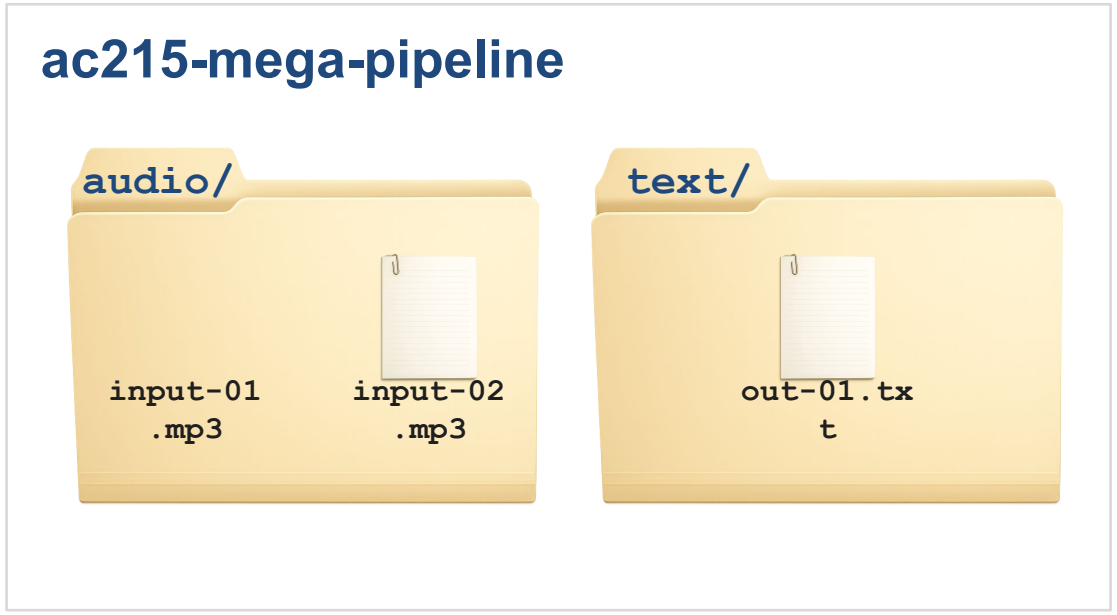
- **Object storage** — files (objects) live in **buckets**: a flat key→blob store, not a filesystem.
- **Global unique address** — `gs://<bucket>/<path>/<object>`
- **Built for MLOps** — durable, cheap, scales endlessly, reachable from any container or service.



# Buckets & Objects

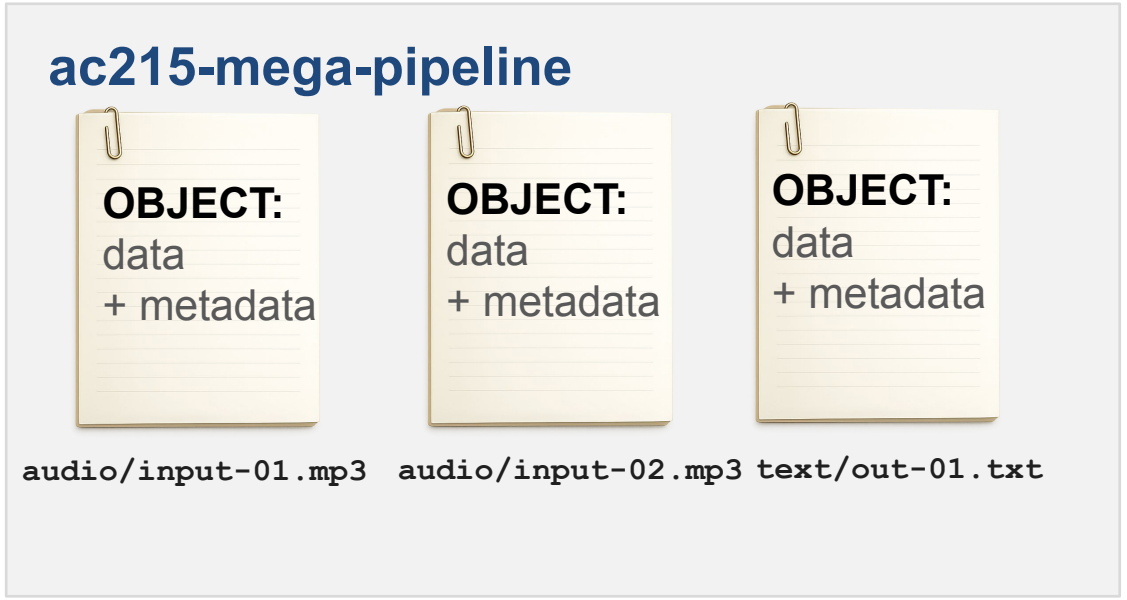
**A bucket holds objects addressed by key — the "folders" are an illusion.**

## How you picture it — a folder tree




Folders really contain files — a hierarchy you open.

## What GCS actually keeps — a flat key space




An object's name points to its data and metadata.

# How a Container Connects to GCS

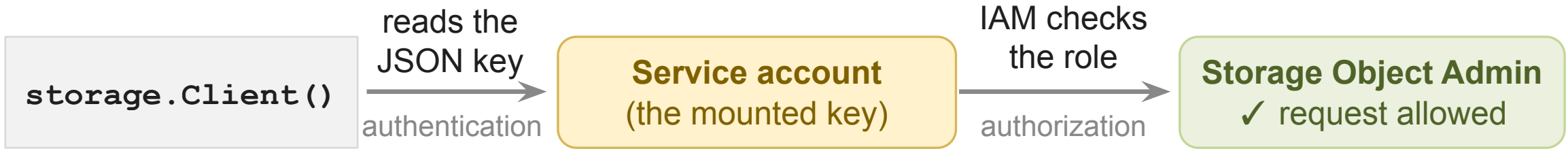
The Service Account JSON key, mounted in with -v.

```
$ docker run --rm -ti \
  -v "$PWD/../secrets":/secrets \
  -e
GOOGLE_APPLICATION_CREDENTIALS=/secrets/data-servic
e-account.json \
  transcribe-audio
```

google-cloud-storage — the client library. (Next Slide)

env var pointing the library at that key.

## What happens when the container runs:



# Reading & Writing Objects

**Authenticates** with the mounted key — no password in code.

```
from google.cloud import storage
client = storage.Client()
bucket = client.bucket("ac215-mega-pipeline")

blob = bucket.blob("audio/input-01.mp3")
blob.upload_from_filename("input-01.mp3")

blob = bucket.blob("text/output-01.txt")
blob.download_to_filename("output-01.txt")

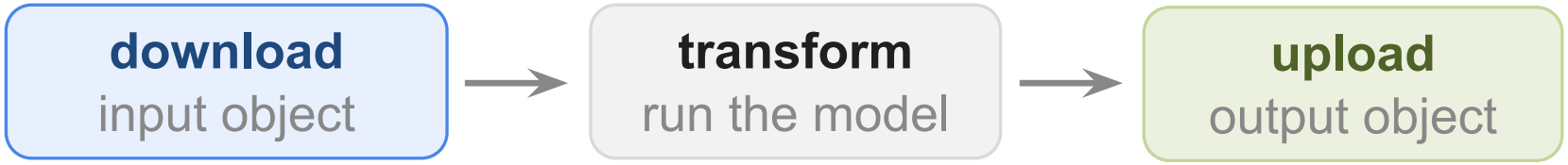
for b in client.list_blobs("ac215-mega-pipeline",
prefix="audio/"):
    print(b.name)
```

**Upload** a local file up to the bucket.

**Download** an object back down to a local file.

**List** objects under a key prefix — like opening a folder.

**Every stage runs the same loop:**



# Same Thing From the CLI

```
$ gcloud storage cp input-01.mp3 \
    gs://ac215-mega-pipeline/audio/

$ gcloud storage cp \
    gs://ac215-mega-pipeline/text/output-01.txt .

$ gcloud storage ls \
    gs://ac215-mega-pipeline/audio/
```

**Upload** — local file → bucket.

**Download** — bucket → local file.

**List** a bucket or prefix — what's in there.

Older **`gsutil cp / ls`** still works — **`gcloud storage`** is the modern command.

# Permissions & Gotchas

**A few rules keep your storage safe, private, and cheap:**

- **Least privilege** — **Storage Object Admin** (read/write objects) is enough; **Storage Admin** also owns buckets. Grant the smaller one.
- **Private by default** — never make a data bucket public; containers reach it through the service account, not the open internet.
- **Names are global** — one worldwide namespace; prefix with the project (`ac215-…`) to avoid collisions.
- **Mind the cost** — you pay to store and to move data out (egress); delete what you don’t need.

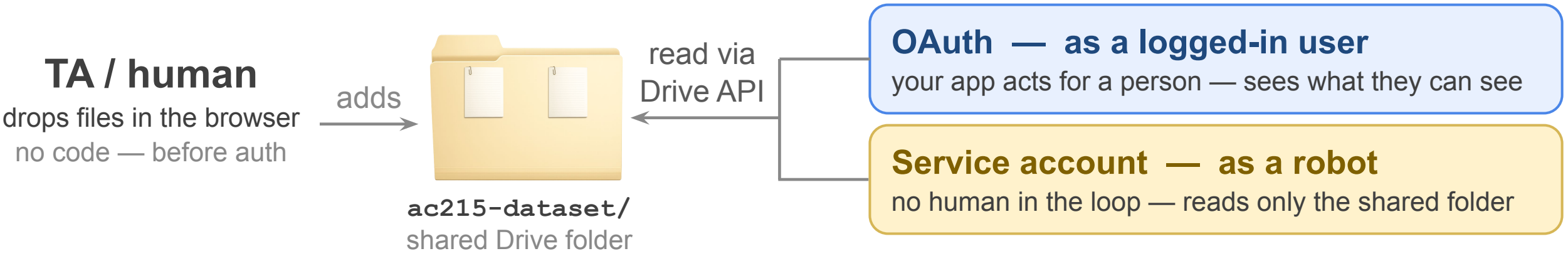
# Outline

- Networking & Ports
- Security and authentication (detour)
- Google Cloud Storage
- **Connecting to Google Drive**
- Tutorial

# Why Connect to Google Drive?

**Sometimes the data lives where the humans are — in Google Drive.**

- **Drive is where people work** — datasets, docs, and recordings land there first; pipelines often need to pull them in by code.



# Drive API Basics

## Drive's own setup — four steps, each with its snippet:

**1 · Enable & install** — enable once in GCP; it's Google's general API client — no google-cloud-drive pkg.

```
$ pip install
google-api-python-client    ✓
$ pip install google-cloud-drive
✗ (no such package)
```

**2 · Build the client** — one import, then a service object.

```
from googleapiclient.discovery
import build

drive = build("drive", "v3",
credentials=creds)
```

**3 · Everything is a field** — the tail of any share link (a folder is just a file of type 'folder').

```
drive.google.com/file/d/1AbC…Xy/view
drive.google.com/drive/folders/1Zz…Qw
```

**4 · Scopes = least privilege** — read-only for pipelines; plain drive (no suffix) adds write — only if truly needed.

```
.../auth/drive.readonly
.../auth/drive
```

# Two Ways to Authenticate to Drive

**Same fork as the auth section: who is acting — a user, or your code?**

| | OAuth 2.0 | Service account |
|---|---|---|
| **Who acts** | a logged-in user | your code — no human |
| **Sees** | that user's own files | only files shared with its email |
| **Best for** | user-facing apps | pipelines · CI · automation |

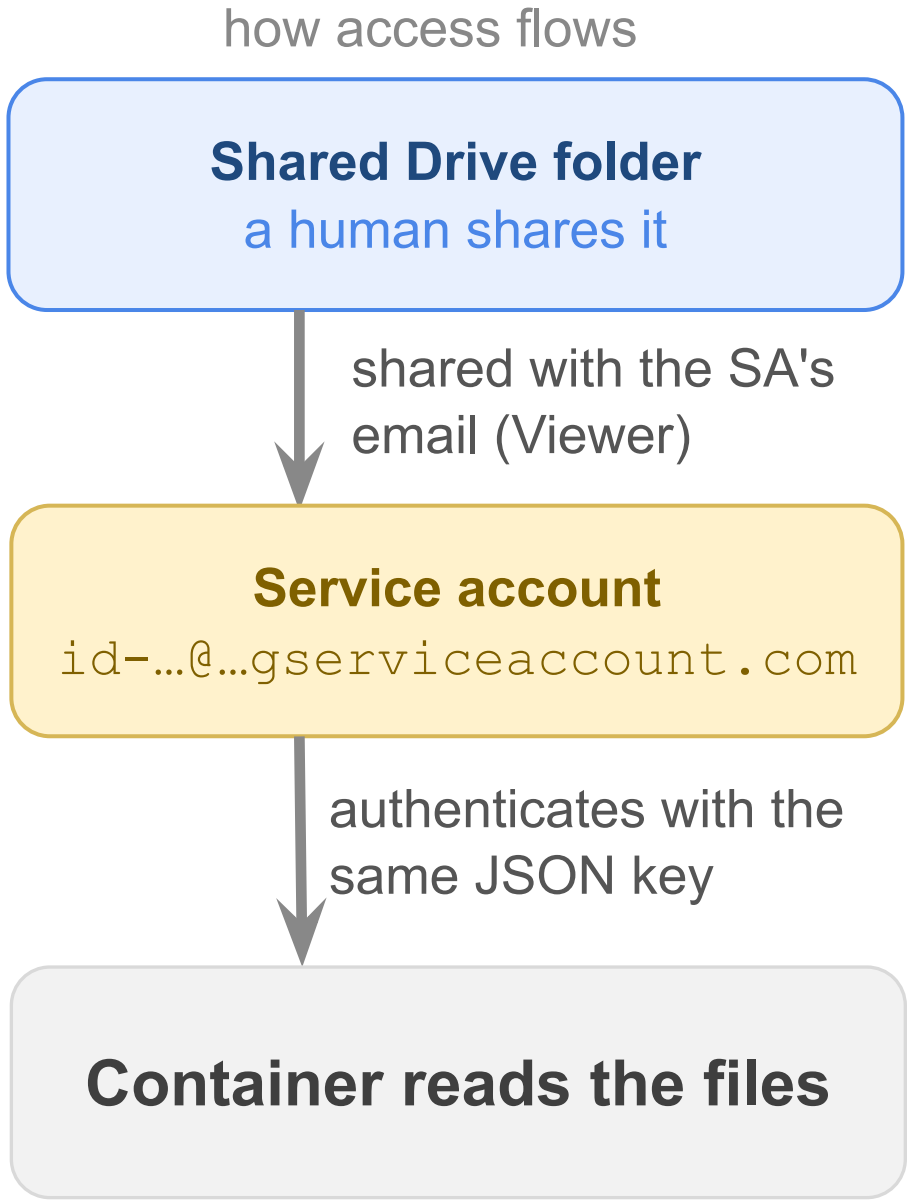
- **A service account has its own (empty) Drive** — it sees a file only after someone shares it with the SA's email. That constraint is the safety feature.
- **Decision:**
  Automation → service account + a shared folder.
  A user's own data → OAuth.

# Service-Account Access Pattern

## The recipe — four steps, one identity:

1. Copy the service account's **email** (`id-…@…gserviceaccount.com`).
2. In Drive, **share the folder with that email** — Viewer is enough.
3. Build a Drive client with the **same SA JSON key** you use for GCS.
4. List & download by the folder's **`fileId`**.

**Same key as GCS — no user login, no new secret. One identity, many Google services.**



# Listing & Downloading

```python
from googleapiclient.discovery import build
from google.oauth2 import service_account
SCOPE = "https://www.googleapis.com/auth/drive.readonly"
creds = service_account.Credentials.from_service_account_file(
        "/secrets/data-service-account.json", scopes=[SCOPE])
drive = build("drive", "v3", credentials=creds)
q = "'<FOLDER_ID>' in parents and trashed=false"
res = drive.files().list(q=q, fields="files(id,name)").execute()
for f in res["files"]:
    print(f["name"], f["id"])
data = drive.files().get_media(fileId="<FILE_ID>").execute()
open("local.pdf", "wb").write(data)
```

**Read-only** — we only pull files, never change them.

**Same key** — the secret you already mount, nothing new.

**List** — ask for one folder's files, skip the trash.

**Download** — grab the raw bytes, then save to disk.

**This is how the course's own `tools/drive_pdf.py` pulls lecture PDFs — real tooling you can read.**

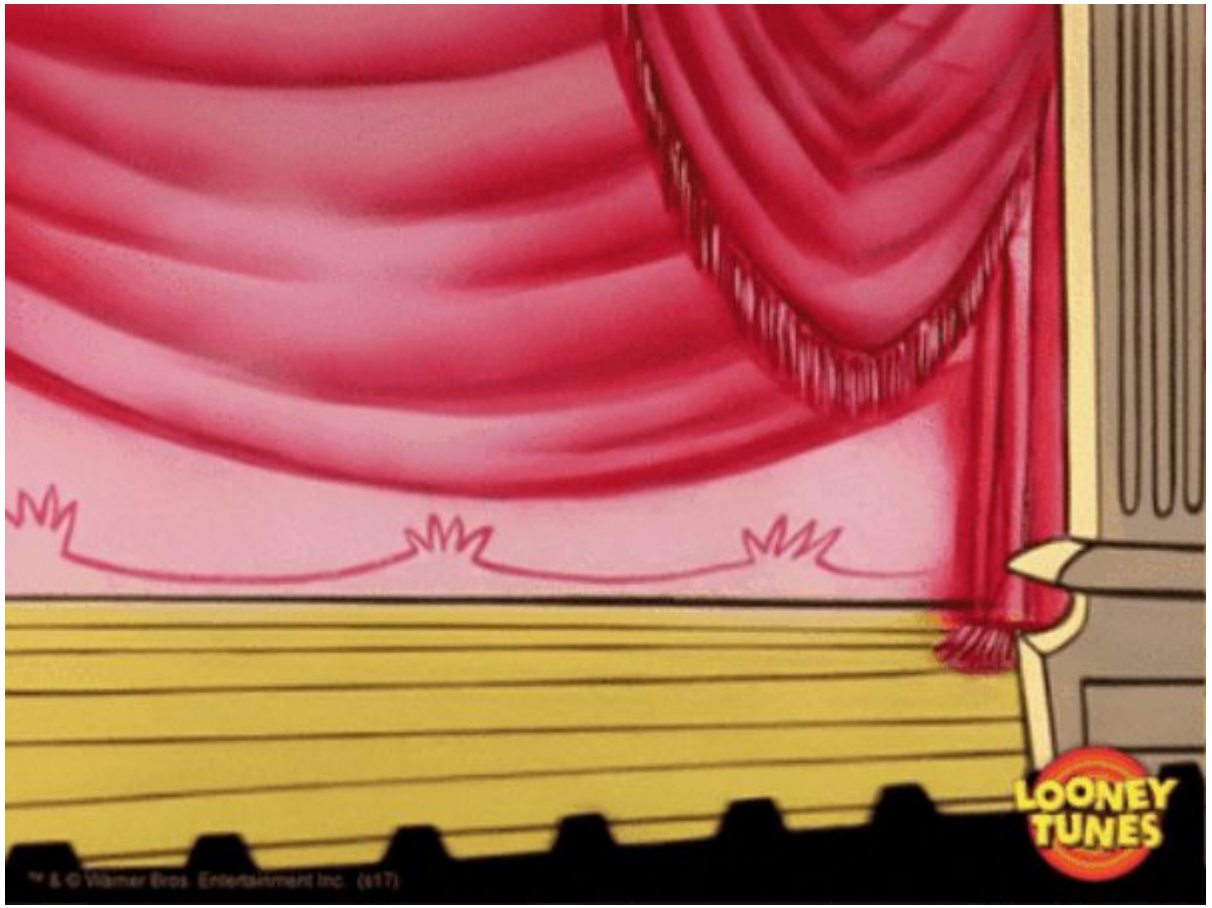
# Outline

- Networking & Ports
- Security and authentication (detour)
- Google Cloud Storage
- Connecting to Google Drive
- **Tutorial**

# Summary: Connecting to the Cloud

- **Networking**: IP addresses identify machines, ports identify services. Docker publishes HOST:CONTAINER ports and resolves service names.
- **Identity**: OAuth grants access on a user's behalf. Service accounts identify code. Federation gives external workloads short-lived tokens.
- **Permissions**: Authentication proves identity. IAM controls allowed actions. Grant only the permissions each workload needs.
- **Cloud Storage:** Buckets hold objects by key. Pipeline stages download inputs, process them, and upload outputs.
- **Google Drive**: Use OAuth for user data or share a folder with a service account. Read files by ID with the minimum required scope.


Delete! Delete! Delete!




# THANK YOU